

Linear models

Assumption and bias

Some ML approaches make **assumptions** about the data generating distribution

- **Right** assumptions → **better performance**
- **Wrong** assumptions → the approach **fails**

The bias of a model **shows how strong the model assumptions are**

- **High** bias → **strong** assumptions
- **Low** bias → **minimal** assumptions

Linear models

A **linear model** assumes the **data is linearly separable**

- The **split** is represented by a **line (2 dimensions) or a hyperplane (3+ dimensions)**

A **linear model** in a n -dimensional space (i.e. n features) is **defined by $n + 1$ weights**

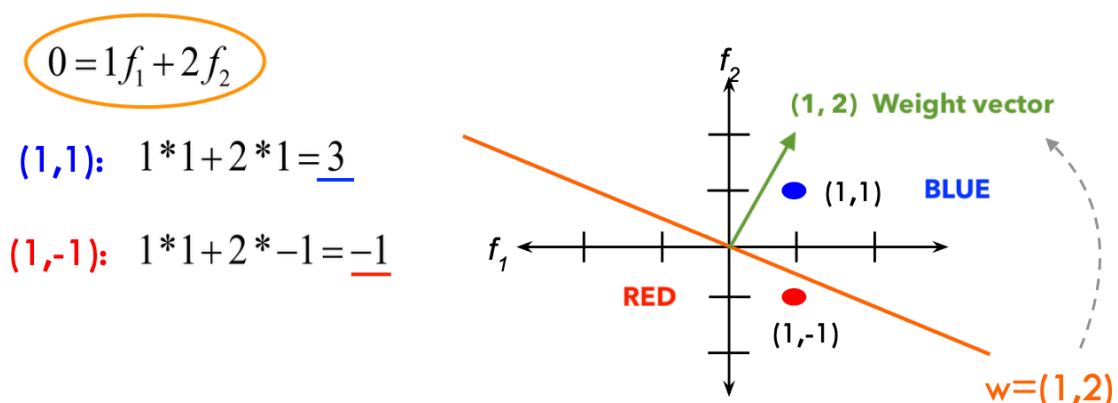
$$0 = w_1 f_1 + w_2 f_2 + b \quad \text{2-dimensions space}$$

$$0 = w_1 f_1 + w_2 f_2 + w_3 f_3 + b \quad \text{3-dimensions space}$$

$$0 = b + \sum_{i=1}^n w_i f_i \quad n\text{-dimensions space}$$

In **2-dimensions space** any pair of values (w_1, w_2) **defines a line**

💡 The value of the bias b changes the coordinates where the line intercepts the axes
(if $b = 0$ it passes through the origin)



The **classification** works by **checking the result sign of the single example**

Online training

As in linear models the **assumption is strong**, online training is commonly used

- The **model interprets data one by one**
 - Receives an example, makes a prediction, if it fails it immediately updates

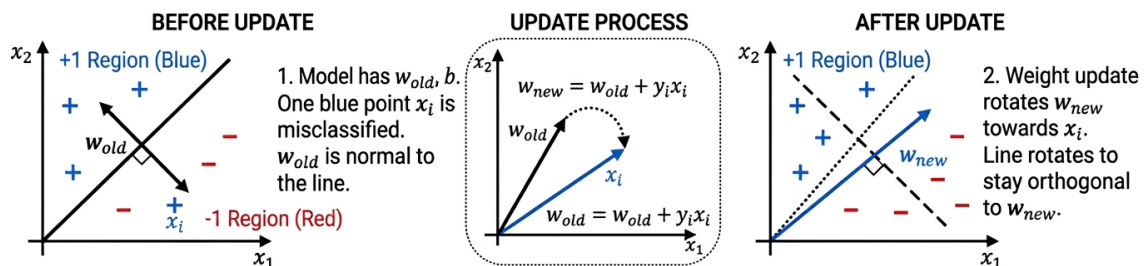
The perceptron algorithm

A classic online learning algorithm

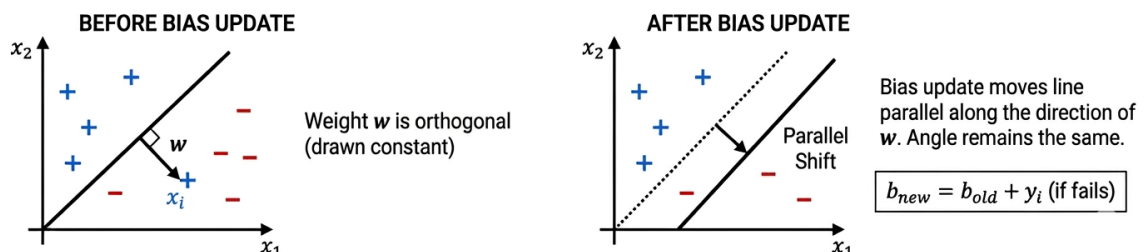
Idea: The **algorithm gets** an **example** as input and **tries to classify it**

- **Correctly predicts** the classification → Nothing
- **Doesn't predict** the classification → The **weights w** and the **bias b** need to be **adjusted**
 - Given:
 - f_i the i -th feature
 - y_i the label (ground truth) for the i -th feature, equals to $+1$ or -1
 - y'_i the prediction of the model for the i -th feature, it isn't correct
 - Applies the following:
 - **Weight adjustment:** $w_i = w_i + (y_i \cdot f_i) \rightarrow$ rotation movement
 - **Bias adjustment:** $b = b + y_i \rightarrow$ vertical movement

PART 1: WEIGHT UPDATE (ROTATION)



PART 2: BIAS UPDATE (TRANSLATION)



Data is read following a **random order**

→ If a **splitting line exists**, the **perceptron** algorithm **finds it**

PERCEPTRON LEARNING ALGORITHM

Algorithm 1 perceptron

```
1: repeat until convergence (or for some # of iterations)
2:   for all training examples  $(f_1, f_2, \dots, f_n, label)$  do
3:     check if it is correct based on the current model
4:     if not correct then
5:       update all the weights:
6:       for all  $w_i$  do
7:          $w_i \leftarrow w_i + f_i \cdot label$ 
8:       end for
9:        $b \leftarrow b + label$ 
10:    end if
11:  end for
12: until convergence
```

💡 The or for some # of iterations conditions makes sure the algorithm ends

- If the dataset isn't linearly separable, convergence isn't guaranteed
- → The algorithm would cycle forever

⚠️ As the algorithm iterates on all the training examples, there is a high risk of overfitting

- We should update the algorithm to only consider random samples of the training set

Algorithm 2 perceptron random sample

```
1: repeat until convergence (or for some # of iterations)
2:   random sample one example  $(f_1, f_2, \dots, f_n, label)$ 
3:   check if it is correct based on the current model
4:   if not correct then
5:     update all the weights:
6:     for all  $w_i$  do
7:        $w_i \leftarrow w_i + f_i \cdot label$ 
8:     end for
9:      $b \leftarrow b + label$ 
10:  end if
11: until convergence
```
